Integrating Feature Confidence and Prior Importance in Deep Learning for Tabular Time-Series and Cross-Sectional Panel Data

Introduction to Feature Weighting in Financial Machine Learning

In the domain of quantitative finance and return prediction, modeling tabular data—comprising both time-series dynamics and cross-sectional panel structures—presents a unique set of structural and statistical challenges. Financial datasets are inherently characterized by extreme non-stationarity, shifting macroeconomic regimes, and exceptionally low signal-to-noise ratios. Traditional deep learning architectures, such as standard Multi-Layer Perceptrons (MLPs) and naive sequence-to-sequence Transformers, were originally designed for domains with high signal density, such as computer vision and natural language processing. In these traditional architectures, all input features are implicitly treated with equal structural priority during the initial embedding phase, relying purely on backpropagation and gradient descent to optimize internal weights over numerous epochs.

However, this unconstrained optimization frequently leads to severe overfitting in financial contexts. When deep networks are permitted to map complex, non-linear relationships without structural guardrails, they inevitably memorize spurious correlations within the market noise rather than identifying true causal drivers of asset returns. A feature that exhibits strong predictive power during a low-volatility bull market (such as price momentum) may become entirely uninformative or actively detrimental during a high-volatility mean-reverting regime. Therefore, to achieve robust generalization and out-of-sample stability in predicting return time series, it is strictly necessary to constrain the hypothesis space.

One of the most rigorous and mathematically sound methodologies for achieving this constraint is the integration of "feature confidence levels" and "feature importance priors" directly into the deep learning algorithm. This paradigm shift transitions the neural network from a passive receiver of high-dimensional data into an active discriminator of feature quality. By implementing these mechanisms, the model is either forced to respect the domain knowledge of the quantitative researcher or endowed with the architectural capacity to dynamically assess the trustworthiness of its own inputs at each specific time step.

Research in this domain bifurcates into three distinct but highly complementary approaches:

1. **Endogenous Dynamic Feature Weighting:** Architectural mechanisms, such as gating

networks and sparse attention masks, designed to dynamically assess the reliability, relevance, or confidence level of a feature on a per-instance or per-time-step basis.
2. **Exogenous Prior Injection:** Objective function modifications, attribution penalties, and Bayesian frameworks that allow researchers to explicitly encode structural, theoretical, or semantic priors regarding which features *should* drive the model's predictions, penalizing the network if it deviates from this domain knowledge.
3. **Uncertainty-Aware Output Modulation:** Methodologies that quantify the heteroscedastic uncertainty of the model's predictions, using the predictive variance as a proxy for feature and model confidence to scale downstream financial allocations dynamically.

This comprehensive report provides an exhaustive analysis of the most rigorous, highly cited, and state-of-the-art methodologies for integrating feature confidence and prior importance into deep neural networks for tabular data. It details the mathematical foundations of Variable Selection Networks, instance-wise attention mechanisms, Axiomatic Attribution Priors, and uncertainty-aware lookahead models, providing a definitive theoretical and practical blueprint for augmenting existing MLP and Transformer architectures.

Endogenous Dynamic Feature Weighting: Architectural Mechanisms

To dynamically assess feature confidence during the forward pass of a neural network, the architecture must contain a localized routing or gating mechanism prior to the primary representation learning layers. This allows the network to suppress noisy, missing, or regime-irrelevant features on a per-instance (cross-sectional) or per-time-step (time-series) basis. This is particularly critical in financial panel data, where the dimensionality of the feature space often exceeds the number of independent observations, making dynamic dimensionality reduction an absolute necessity.

The Variable Selection Network and Temporal Fusion Transformers

One of the most foundational and highly cited advancements in tabular time-series modeling is the Temporal Fusion Transformer (TFT), introduced by researchers Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister.[1] The defining innovation of the TFT for tabular data is its Variable Selection Network (VSN), which provides an exact mathematical framework for dynamic feature confidence and selection.[5]

The VSN operates on the principle that the relevance of dynamic inputs (e.g., technical indicators, macroeconomic variables, high-frequency order book imbalances) changes dynamically over time, while the relevance of static inputs (e.g., sector classifications, market capitalization brackets, geographic regions) depends on the cross-sectional entity being

evaluated. The VSN performs feature-wise non-linear embeddings followed by a dynamic soft selection of relevant covariates at each time step, effectively computing a continuous confidence score for every input.[8]

Mathematically, the VSN utilizes Gated Residual Networks (GRNs) as its foundational building block. A GRN provides the flexibility to apply non-linear processing only where necessary, utilizing a skip connection to bypass the transformation if the dataset does not require complex routing.[9] For a given input vector a and a static context vector c (which provides cross-sectional conditioning), the GRN is defined as:

$$GRN_{\omega}(a, c) = LayerNorm\left(a + GLU_{\omega}(W_1 a + W_2 c + b_1)\right)$$

where GLU represents a Gated Linear Unit, which controls the flow of information by acting as a dynamic filter.[10] The GLU suppresses signals that are deemed uninformative based on the current context, which is a highly effective mechanism for managing the low signal-to-noise ratio in financial returns. In the context of the VSN, an individual, independent GRN is applied to each feature j at time step t, producing a feature-specific encoded representation $\xi_t^{(j)}$.[7]

To calculate the dynamic feature confidence (or selection weight) for each individual feature, a global GRN is applied to the concatenated input vector of all flattened features, which is then passed through a Softmax activation layer:

$$v_t = Softmax\left(GRN_{vsn}(\xi_t^{(1)}, \xi_t^{(2)}, \ldots, \xi_t^{(m_x)})\right)$$

The output v_t is a vector of feature confidence scores $\alpha_{t,j}$, where the sum of the weights across all features equals one ($\sum_j \alpha_{t,j} = 1$). The final representation passed to the subsequent sequence-to-sequence model or Multi-Head Attention layers is the weighted sum of the transformed features:

$$\tilde{x}_t = \sum_{j=1}^{m_x} \alpha_{t,j} W_j \xi_t^{(j)}$$

The application of this architecture to return prediction yields immediate benefits for interpretability and regularization. Integrating a VSN prior to a standard MLP or Transformer model effectively serves as an endogenous feature confidence module that requires no manual hyperparameter tuning for feature dropping. By analyzing the output $\alpha_{t,j}$ weights, researchers can extract the precise dynamic importance the model assigns to specific predictors at specific times, offering profound insights into regime shifts.[5] The VSN automates

variable selection and filters out noise, which is imperative in quantitative finance where incorporating redundant or low-confidence features drastically degrades out-of-sample performance and exacerbates alpha decay.[6]

Instance-Wise Feature Selection via TabNet

While Variable Selection Networks are highly effective for sequential time-series data, the TabNet architecture, developed by Sercan Ö. Arik and Tomas Pfister, introduces a rigorous framework for instance-wise feature selection that is exceptionally powerful for cross-sectional panel data.[14]

TabNet diverges fundamentally from global feature selection algorithms (such as Random Forests or standard Lasso regression, where features are selected identically for all data points across the entire dataset). Instead, it applies instance-wise selection, dynamically choosing which features to process based entirely on the specific values of the input instance itself.[16] It achieves this through a sequential multi-step architecture, operating fundamentally differently from standard MLPs by mimicking the decision-making process of an ensemble of trees but retaining the gradient-descent optimization of a neural network.[14]

At each decision step i within the TabNet architecture, the model employs an attentive transformer block to generate a trainable mask $M[i]$. This mask acts as the absolute feature confidence level for that specific forward pass. Crucially, TabNet utilizes the sparsemax activation function rather than the standard softmax used in the TFT.[18] The sparsemax function projects the input onto the probability simplex, but unlike softmax, which always retains a long tail of infinitesimal probabilities, sparsemax can output exact zeros. This enforces true, hard sparsity rather than soft approximations, completely eliminating the computational and statistical influence of features that the network deems uninformative for a specific cross-section.[15]

The attention mask at step i is mathematically formulated as:

$$M[i] = sparsemax\left(P[i-1] \cdot h_i(a[i-1])\right)$$

where h_i is a trainable linear function applied to the output of the previous step, and $P[i-1]$ is a critical prior scale term. The prior scale term explicitly discourages the network from repeatedly selecting features that have already been heavily utilized in previous decision steps.[14] The prior scale is updated multiplicatively after each step:

$$P[i] = P[i-1] \cdot (\gamma - M[i])$$

where γ is a relaxation parameter. By multiplying the raw input features by this mask ($X \odot M[i]$), the network explicitly filters out low-confidence features before they are passed into the feature transformer blocks for non-linear processing.[15]

For researchers utilizing cross-sectional models to predict asset returns, embedding a TabNet-style sequential masking mechanism acts as a robust, non-linear feature confidence filter. If the model determines that a specific cross-sectional entity (e.g., a micro-cap technology stock) exhibits financial characteristics where macroeconomic interest rate features are irrelevant compared to idiosyncratic momentum, the sparsemax mask will force the weights of the macroeconomic features to exactly zero. This isolates the true signal from idiosyncratic noise on a per-asset basis, a capability that global feature selection techniques inherently lack.[17]

Architectural Mechanism	**Primary Activation for Confidence**	**Type of Sparsity Enforced**	**Optimal Financial Application**
Variable Selection Network (VSN)	Softmax over Gated Residual Networks	Soft Selection (Weights sum to 1, rarely exact zero)	Time-series forecasting, capturing smooth macroeconomic regime shifts.
TabNet Sequential Masking	Sparsemax over Attentive Transformers	Hard Sparsity (Exact zeros for irrelevant features)	Cross-sectional equity screening, instance-wise fundamental analysis.

Transformer-Native Feature Confidence and Tokenization

As the deep learning landscape has evolved, researchers have sought ways to adapt the native mechanisms of the Transformer architecture—specifically Multi-Head Self-Attention (MHSA)—to tabular data. This approach avoids the need for external gating networks by forcing the Transformer to learn feature confidence through its internal self-attention maps.

FT-Transformer: Feature Tokenization as Importance

Yury Gorishniy et al. introduced the FT-Transformer (Feature Tokenizer + Transformer), which adapts the Transformer architecture specifically for tabular domains.[19] Rather than using routing networks (like VSN) or sparse masks (like TabNet), the FT-Transformer natively leverages the self-attention mechanism to derive feature confidence and map complex, higher-order feature interactions.[20]

The core structural innovation that separates the FT-Transformer from a standard MLP is the Feature Tokenizer. In standard MLPs, tabular features are simply concatenated into a single, static vector. In the FT-Transformer, each individual feature—whether continuous (e.g., historical volatility) or categorical (e.g., exchange code)—is linearly embedded into a uniform discrete token representation $T_j \in \mathbb{R}^d$.[20] A special `` (classification/regression) token is appended to this sequence of feature tokens, and the entire set is passed through a stack of standard Transformer blocks.[22]

The feature confidence level in the FT-Transformer is directly accessible and interpretable via the attention map of the token across all layers and heads.[23, 24] Because the Transformer models pairwise interactions between every single feature token, the attention weights connecting the feature tokens to the token effectively quantify the relative importance of each feature dynamically for each sample.[19] Empirical studies, including those by Gorishniy et al., have demonstrated that the feature importance derived from these attention maps exhibits a remarkably high rank correlation with rigorous post-hoc attribution methods like Integrated Gradients, confirming that the attention mechanism is a valid proxy for dynamic feature confidence.[19]

Feature-Missing-Aware Calibration and Confidence Scoring

In many real-world tabular datasets, particularly in finance and healthcare, feature confidence is directly correlated with data sparsity or missingness. Corporate fundamentals are reported quarterly, while price data is reported daily; macroeconomic indicators suffer from reporting lags and subsequent revisions. Conventional imputation methods (such as forward filling or mean imputation) are highly problematic because they treat real, observed data and artificially imputed data identically during the forward pass of the neural network.

Recent research has addressed this by explicitly incorporating a Feature Confidence Learner Module directly into the architecture. Zhu et al. developed PRISM (and related frameworks like EMERGE), which computes a continuous confidence score for every feature based on its temporal distance from a true observation.[25]

Within this framework, a feature confidence score $c_{i,j} \in $ is calculated for every feature j of instance i based on its missing status.[25] This confidence score is then integrated directly into an attention mechanism to produce calibrated hidden representations. The standard attention weights α_i are explicitly penalized or scaled down to α_i^* when the feature confidence $c_{i,j}$ is low, ensuring the network does not over-rely on imprecise, stale, or imputed values.[27] For financial panel data, implementing a temporal-decay feature confidence module prevents the Transformer from treating stale, forward-filled quarterly earnings data with the same structural confidence as real-time, high-frequency price data.

Exogenous Prior Injection: Regularizing with Domain Knowledge

While endogenous mechanisms like VSNs and FT-Transformers allow the network to dynamically assign confidence based on the data, financial datasets often lack the sheer signal density required for the network to *learn* the correct confidence weights entirely from scratch. Deep learning models operating on financial returns frequently suffer from the "Clever Hans" effect—they learn spurious noise artifacts or exploit data leakage rather than mapping true causal economic drivers.

To counteract this, machine learning researchers must inject "feature importance priors." This involves explicitly instructing the model prior to, or during, the fitting process regarding which features are inherently more trustworthy based on economic theory or empirical historical research. This is achieved via sophisticated objective function regularization.

Axiomatic Attribution Priors

The most rigorous, mathematically sound, and highly cited methodology for injecting explicit feature importance priors into deep neural networks is the framework of **Axiomatic Attribution Priors**, introduced by Gabriel Erion, Joseph D. Janizek, Pascal Sturmfels, Scott M. Lundberg, and Su-In Lee (published in *Nature Machine Intelligence*, 2021).[28]

This framework operates on the foundational premise that the importance a model assigns to a feature can be robustly quantified using attribution methods, specifically Shapley values (SHAP) or Expected Gradients. These methods satisfy critical game-theoretic axioms: Efficiency (the sum of attributions equals the model output), Symmetry (identical features receive identical attributions), Dummy (features with no impact receive zero attribution), and Additivity.[31] The researchers demonstrated a profound breakthrough: one can optimize neural networks to not only minimize standard prediction error but also to simultaneously align the model's internal feature attributions with a predefined, human-engineered prior of feature importance.[28]

Let $f_\theta(x)$ be the neural network. An attribution method $\phi_j(f_\theta, x)$ returns the marginal contribution of feature j to the specific prediction $f_\theta(x)$. The researcher defines a prior matrix or vector π_{ij}, representing the desired absolute importance or confidence level for feature j on sample i.[35]

The model is then trained with a composite loss function that balances task accuracy with prior alignment:

$$\mathcal{L}(\theta) = \mathcal{L}_{task}(f_\theta(X), Y) + \lambda\Omega(\phi(f_\theta, X), \pi)$$

where $\mathcal{L}_{task}$ is the standard forecasting loss (e.g., Mean Squared Error or Huber loss for regression), λ is a tuning hyperparameter controlling the strength of the prior, and Ω is a penalty function measuring the divergence between the model's actual feature attributions and the intended prior π.[35]

Calculating true, exact SHAP values at every forward pass of a training loop is computationally impossible, as it scales exponentially with the number of features ($O(2^F)$). To solve this, Erion et al. propose using **Expected Gradients** as a fast, differentiable approximation of Shapley values during training.[28] Expected Gradients are calculated by integrating the gradients of the neural network's output with respect to the input over a continuous path from a baseline reference distribution x' to the actual input instance x.[31]

The application of Axiomatic Attribution Priors to return prediction is immensely powerful. In quantitative finance, practitioners almost always possess strong theoretical priors. For example, it is widely accepted in the asset pricing literature that long-term fundamental value factors (e.g., Book-to-Market, Earnings Yield) are more robust and grounded in economic reality than short-term, highly parameterized technical indicators. By setting the prior vector π_j high for fundamental features and low for high-frequency noise, the Axiomatic Attribution Prior forces the neural network to exhaust the predictive power of the fundamental features before it is mathematically permitted to rely on the technical indicators.[28] If the network attempts to overfit to a noise feature to minimize training loss, the attribution ϕ_j for that noise feature will spike, triggering a massive penalty via $\lambda\Omega$, forcing the gradient descent optimizer to find a solution that respects the financial domain knowledge.[28]

Feature Attribution Distance Loss

In a closely related line of research, Liu and Avci (2019) proposed incorporating priors by adding a direct L2 distance loss between feature attributions and task-specific prior values into the objective function.[35]

While originally applied to natural language processing models to neutralize societal bias, the mathematical framework transfers directly and seamlessly to tabular machine learning. Given an input vector x of length n, and an attribution target vector t where t_j is the target importance (the explicit prior) for the j-th feature, the prior loss is defined straightforwardly as:

$$\mathcal{L}_{prior}(\phi(x), t) = \sum_{j=1}^{n} (\phi_j(x) - t_j)^2$$

This formulation provides a continuous, highly differentiable mechanism to enforce feature confidence.[35] If a machine learning researcher possesses an MLP or Transformer that is generally reliable but wishes to guarantee that the model prioritizes specific inputs (for example, ensuring that VIX levels act as a primary conditioning variable in all forecasts), adding this L2 attribution loss to the PyTorch or TensorFlow training loop acts as a strict, unbreakable structural guardrail against feature neglect.

Statistical and Bayesian Prior Ensembles

A critical vulnerability in the methodology of exogenous prior injection is the risk of prior misspecification. What happens if the quantitative researcher's prior is empirically incorrect? If a researcher forces a neural network to rely heavily on a specific financial feature that has actually lost its predictive edge due to alpha decay or structural market changes, standard regularization techniques (like the L2 attribution penalty) will force the model to conform to the erroneous prior, severely degrading out-of-sample performance.

To resolve this fragility, contemporary research has developed frameworks that treat priors probabilistically, allowing the model to adaptively trust or discard the prior based on its statistical alignment with the empirical data.

Statsformer and Oracle-Guaranteed Semantic Priors

A highly contemporary advancement in prior injection is the **Statsformer** framework (Zhang et al., 2026), which formalizes the integration of external semantic priors (often derived from Large Language Models, alternative data sources, or domain experts) into tabular supervised learning.[38]

Statsformer utilizes a model-agnostic aggregation framework designed with rigorous

oracle-style statistical guarantees.[38] It accepts a semantic prior vector $V = (v_1, \ldots, v_p)$ where v_j represents the prior relevance or expected confidence of feature j. Rather than forcing the model to strictly adhere to V through rigid penalties, Statsformer defines a monotone, temperature-controlled family of mapping functions $\tau : [0, \infty) \to (0, \infty)$, scaled by a parameter α.[40]

The prior score is transformed via the mapping function $\tau(v_j)$ before being injected into the learning procedure as an adaptive feature weight w_j. Through classical convex aggregation theory, the Statsformer framework mathematically guarantees that if the prior is informative and aligned with the underlying data generating process, the model achieves consistent, measurable performance improvements over standard baselines. Crucially, if the prior is uninformative, misspecified, or "hallucinated" (in the case of LLM-derived priors), the statistical framework automatically downweights the external guidance during the ensemble aggregation phase, entirely mitigating the impact of erroneous priors.[38]

Bayesian Neural Networks and Horseshoe Priors

A mathematically distinct, yet deeply related approach to integrating feature importance priors is the application of Bayesian Neural Networks (BNNs). Rather than utilizing expected gradients or deterministic attention masks, BNNs place continuous probability distributions over all the weights of the neural network.[42] This allows the network to model its own epistemic uncertainty regarding feature importance.

To perform rigorous feature selection and enforce feature importance priors within a Bayesian framework, researchers frequently apply **Horseshoe Priors** or similar continuous shrinkage priors specifically to the weights incident to the input nodes of the network.[42] A horseshoe prior is a heavy-tailed statistical distribution characterized by an infinite spike of density exactly at zero, combined with heavy tails that extend outward.

By applying a horseshoe prior to the variance of the weights connected to input feature j, the Bayesian inference process acts as a continuous, differentiable relaxation of discrete spike-and-slab feature selection.[42] If the researcher's domain prior dictates that a specific feature is of low confidence or minimal theoretical importance, the hyper-parameters of the horseshoe prior corresponding to that feature are adjusted to exert extreme statistical shrinkage.

During the inference process—typically executed via Variational Inference or parallel tempering Markov Chain Monte Carlo (MCMC)—the optimization will naturally shrink the weights of

low-confidence features exactly to zero, effectively "turning off" the input node entirely. Simultaneously, it allows the weights of high-confidence features to escape shrinkage entirely due to the heavy tails of the horseshoe distribution.[42] This approach provides a mathematically elegant, highly rigorous way to embed domain knowledge as probabilistic priors, ensuring that the neural network's structural complexity is inherently tied to the researcher's absolute confidence in the input features.[43] BNNs have been shown to be exceptionally resilient in modeling stock price forecasting during periods of extreme market volatility, such as the COVID-19 pandemic, where traditional deterministic MLPs fail catastrophic due to out-of-distribution feature shifts.[45]

Prior Injection Methodology	**Mathematical Mechanism**	**Handling of Misspecified Priors**	**Optimal Use Case**
Axiomatic Attribution Priors	Expected Gradients + L2 Penalty	Vulnerable; model is forced to adhere to the prior.	When theoretical domain knowledge is absolutely certain.
Statsformer Framework	Monotone mapping + Convex Aggregation	Highly Robust; automatically downweights erroneous priors.	When priors are derived from LLMs or noisy alternative data.
Bayesian Horseshoe Priors	Continuous Shrinkage Distributions via MCMC/VI	Robust; data likelihood can overwhelm a weak prior.	When epistemic uncertainty quantification is strictly required.

Uncertainty-Aware Confidence Modulators in Financial Forecasting

While endogenous feature selection mechanisms (like VSNs) and exogenous prior injection manage the *inputs* and the internal *routing* of the neural network, uncertainty-aware mechanisms manage the *outputs* and intermediate representations. They achieve this by quantifying the probabilistic confidence of the data itself, allowing the model to dynamically

scale its reliance on a feature or prediction based on real-time variance.

Uncertainty-Aware Lookahead Factor Models

In the highly specific context of quantitative investing and tabular forecasting, a seminal contribution to this sub-field is the paper "Uncertainty-Aware Lookahead Factor Models for Quantitative Investing" by Lakshay Chauhan, John Alberg, and Zachary Lipton, published at the International Conference on Machine Learning (ICML) in 2020.[46]

Chauhan et al. identified a fundamental flaw in the standard machine learning pipeline for finance: forecasting financial returns directly with deep neural networks (like LSTMs or MLPs) almost universally results in massive overfitting due to the extreme noise and heteroscedasticity inherent in daily price data.[46] To circumvent this, they propose an architectural shift: predicting future fundamental data (e.g., EBIT, revenue, free cash flow) instead of prices, and using these fundamental predictions as inputs to construct financial factors.

Crucially, they introduce a highly robust mechanism for establishing **feature confidence via uncertainty quantification**. Their model does not simply output a deterministic point estimate for a future feature; rather, it is designed to output both the predicted feature value *and* its conditional variance (representing heteroscedastic uncertainty).[46]

They achieve this via two synchronized methods:

1. **Neural Heteroscedastic Regression:** The final layer of the network is bifurcated. It is trained to output both a mean $\mu(x)$ and a variance $\sigma^2(x)$ by minimizing the negative log-likelihood of a Gaussian distribution:

$$\mathcal{L} = \frac{\log \sigma^2(x)}{2} + \frac{(y - \mu(x))^2}{2\sigma^2(x)}$$

2. **MC-Dropout:** Monte Carlo dropout is utilized at inference time. By keeping dropout active during the forward pass and running the data through the network multiple times, the framework generates a distribution of outputs, explicitly capturing the epistemic model uncertainty.[46]

Once the model predicts the future fundamental feature (e.g., expected EBIT) and its associated uncertainty (σ^2), the framework integrates this confidence level directly into the downstream financial application. For instance, a traditional quantitative value factor like EBIT-to-Enterprise Value (EBIT/EV) is algorithmically transformed into an uncertainty-adjusted lookahead factor:

$$\text{Factor Score} = \frac{\mathbb{E}}{\sigma^2 \cdot EV}$$

By placing the predictive variance directly in the denominator of the factor equation, the model explicitly and mathematically down-weights predictions where the network lacks confidence.[46] Companies or time-series data points that are highly unpredictable (resulting in a high σ^2) are algorithmically penalized, essentially reducing their feature importance to near zero for that specific instance.

In rigorous backtests leveraging industry-grade portfolio simulators, this uncertainty-aware feature weighting increased simulated compound annualized returns from 14.0% (for standard deterministic factor models) to 17.7%, and dramatically improved the risk-adjusted Sharpe ratio from 0.52 to 0.84.[46] For an ML researcher, modifying the MLP output head to produce heteroscedastic variance and using it to dynamically weight the feature inputs for a secondary return-prediction model is an exceptionally rigorous, state-of-the-art approach to dynamic confidence modeling.

Architectural Synthesis and Implementation Strategy

For a machine learning researcher who already possesses a reliable, well-tuned MLP and Transformer model for tabular return prediction, successfully integrating "feature confidence levels" and "prior inputs" requires a careful synthesis of the methodologies detailed throughout this report. Adopting a modular approach allows for the combination of endogenous dynamic weighting, exogenous prior regularization, and output uncertainty scaling without completely rebuilding the existing codebase.

1. Augmenting the Input Layer with a Variable Selection Network (VSN)

Rather than passing raw, normalized tabular data directly into the MLP or Transformer embedding layer, the input should first route through a dedicated Variable Selection Network, as pioneered by the Temporal Fusion Transformer architecture.[1]

- **Implementation:** Construct independent Gated Residual Networks (GRNs) for every continuous and categorical feature. Pass the outputs through a global GRN and a Softmax layer to generate dynamic feature weights $\alpha_{t,j}$.[6]
- **Result:** The model natively generates a time-varying, instance-specific confidence score for every feature. This effectively identifies regime shifts in the financial time series where specific factors (momentum, value, volatility) rotate as the dominant predictive drivers, suppressing noise dynamically.[8]

2. Imposing Axiomatic Attribution Priors on the Objective Function

To ensure the newly implemented VSN and the subsequent Transformer layers do not overfit to noise—and to explicitly inject the researcher's prior knowledge regarding feature importance—the standard loss function must be augmented.[28]

- **Implementation:** Define a target prior vector π representing the desired relative importance of the features (e.g., assigning a higher weight to proven fundamental ratios and a lower weight to highly volatile sentiment scores).[36] During training, compute the Expected Gradients (a fast, differentiable approximation of Shapley values) of the network's output with respect to the inputs. Add an L2 penalty term to the loss function that calculates the divergence between the Expected Gradients and the target prior π.[29]
- **Result:** The neural network is structurally forced to utilize the high-confidence features defined by the researcher. It can only rely on lower-confidence features if they provide a predictive advantage so massive that it overcomes the tuning penalty parameter.[36]

3. Implementing Uncertainty-Aware Down-weighting

To manage confidence at the ultimate prediction level, the final layer of the MLP or Transformer should be bifurcated to output both a point prediction μ and a conditional variance σ^2, implementing Neural Heteroscedastic Regression.[46]

- **Implementation:** Use a negative log-likelihood loss function to train the variance head. When constructing the final trading signal, portfolio allocation, or expected return matrix, scale the point prediction μ by the inverse of the variance σ^2.[46]
- **Result:** The model mathematically acknowledges its own epistemic and aleatoric uncertainty, automatically suppressing signals and reducing exposure when feature confidence breaks down due to out-of-distribution market shocks or unprecedented macroeconomic events.[46]

By synthesizing a Variable Selection Network at the input layer, constraining the gradient descent optimization process with Axiomatic Attribution Priors, and utilizing heteroscedastic variance for output signal scaling, a machine learning researcher can transform a standard deep learning model into a heavily regularized, highly interpretable, and exceptionally robust tabular forecasting engine. This comprehensive approach guarantees that the architecture fundamentally understands *what* features to trust, *when* to trust them across changing market regimes, and *why* they are being prioritized, ultimately leading to vastly superior generalization and alpha retention in out-of-sample financial forecasting.

Works cited

1. Time Series Forecasting with the NVIDIA Time Series Prediction Platform and Triton Inference Server | NVIDIA Technical Blog, accessed March 17, 2026, https://developer.nvidia.com/blog/time-series-forecasting-with-the-nvidia-time-series-prediction-platform-and-triton-inference-server/
2. Time Series Forecasting in Financial Markets: Benchmarking the Temporal Fusion Transformer Against N-BEATS. - CCT ARC - CCT College Dublin, accessed March 17, 2026, https://arc.cct.ie/cgi/viewcontent.cgi?article=1154&context=ict
3. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting, accessed March 17, 2026, https://www.personal.soton.ac.uk/cz1y20/Reading_Group/mlts-2023/week3/TFT.pdf
4. Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting, accessed March 17, 2026, https://www.researchgate.net/publication/338064770_Temporal_Fusion_Transformers_for_Interpretable_Multi-horizon_Time_Series_Forecasting
5. Temporal Fusion Transformer: Time Series Forecasting with Deep Learning - Complete Tutorial | Towards Data Science, accessed March 17, 2026, https://towardsdatascience.com/temporal-fusion-transformer-time-series-forecasting-with-deep-learning-complete-tutorial-d32c1e51cd91/
6. Implementation of the Temporal Fusion Transformer for Demand Forecasting - UPCommons, accessed March 17, 2026, https://upcommons.upc.edu/bitstreams/018beb55-8a86-40f4-ab61-0cc3a357309d/download
7. Quantum Temporal Fusion Transformer - arXiv.org, accessed March 17, 2026, https://arxiv.org/html/2508.04048v1
8. Deep Learning for Financial Time Series: A Large-Scale Benchmark of Risk-Adjusted Performance1footnote 11footnote 1Code is available upon request. - arXiv.org, accessed March 17, 2026, https://arxiv.org/html/2603.01820v1
9. Master the State-of-the-Art: coding a Temporal Fusion Transformer by Hand | by Marcel Boersma PhD - Medium, accessed March 17, 2026, https://medium.com/@marcelboersma/temporal-fusion-transformers-from-scratch-a6c10866ccd6
10. PHTFNet-RPM: a probabilistic hybrid network with RPM for tobacco root disease forecasting - Frontiers, accessed March 17, 2026, https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1705587/full
11. Automatic Modulation Classification Using Gated Recurrent Residual Network | Request PDF - ResearchGate, accessed March 17, 2026, https://www.researchgate.net/publication/340989616_Automatic_Modulation_Classification_Using_Gated_Recurrent_Residual_Network
12. Interpretable Time Series Forecasting Using a Temporal Fusion Transformer - MATLAB & Simulink - MathWorks, accessed March 17, 2026, https://www.mathworks.com/help/deeplearning/ug/time-series-forecasting-using

-temporal-fusion-transformer.html
13. Variable Selection Networks - Emergent Mind, accessed March 17, 2026, https://www.emergentmind.com/topics/variable-selection-network
14. TabNet: Attentive Interpretable Tabular Learning - AAAI Publications, accessed March 17, 2026, https://ojs.aaai.org/index.php/AAAI/article/download/16826/16633
15. TabNet: Attentive Interpretable Tabular Learning - arXiv.org, accessed March 17, 2026, https://www.arxiv.org/pdf/1908.07442v4
16. TabNet: A Deep Learning Breakthrough for Tabular Data | by Dong-Keon Kim - Medium, accessed March 17, 2026, https://medium.com/@kdk199604/tabnet-a-deep-learning-breakthrough-for-tabular-data-bcd39c47a81c
17. TabNet - GeeksforGeeks, accessed March 17, 2026, https://www.geeksforgeeks.org/machine-learning/tabnet/
18. Implementing TabNet in PyTorch | Towards Data Science, accessed March 17, 2026, https://towardsdatascience.com/implementing-tabnet-in-pytorch-fc977c383279/
19. The Role of Feature Interactions in Graph-based Tabular Deep Learning - arXiv, accessed March 17, 2026, https://arxiv.org/html/2510.04543v2
20. Revisiting Deep Learning Models for Tabular Data - OpenReview, accessed March 17, 2026, https://openreview.net/pdf?id=i_Q1yrOegLY
21. yandex-research/rtdl-revisiting-models: (NeurIPS 2021) Revisiting Deep Learning Models for Tabular Data - GitHub, accessed March 17, 2026, https://github.com/yandex-research/rtdl-revisiting-models
22. Multi-branch of Attention Yields Accurate Results for Tabular Data - arXiv, accessed March 17, 2026, https://arxiv.org/html/2502.12507v3
23. PRISM: Leveraging Prototype Patient Representations with Feature-Missing-Aware Calibration for EHR Data Sparsity Mitigation - arXiv, accessed March 17, 2026, https://arxiv.org/html/2309.04160v5
24. CIKM '24: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, accessed March 17, 2026, https://www.sigweb.org/toc/cikm24.html
25. (PDF) PRISM: Leveraging Prototype Patient Representations with Feature-Missing-Aware Calibration for EHR Data Sparsity Mitigation - ResearchGate, accessed March 17, 2026, https://www.researchgate.net/publication/373822447_PRISM_Leveraging_Prototype_Patient_Representations_with_Feature-Missing-Aware_Calibration_for_EHR_Data_Sparsity_Mitigation
26. Improving performance of deep learning models with axiomatic attribution priors and expected gradients | Request PDF - ResearchGate, accessed March 17, 2026, https://www.researchgate.net/publication/352010330_Improving_performance_of_deep_learning_models_with_axiomatic_attribution_priors_and_expected_gradients
27. Fast Axiomatic Attribution for Neural Networks - NIPS, accessed March 17, 2026, https://proceedings.neurips.cc/paper/2021/file/a284df1155ec3e67286080500df36a9a-Paper.pdf

28. Four Axiomatic Characterizations of the Integrated Gradients Attribution Method - Journal of Machine Learning Research, accessed March 17, 2026, http://www.jmlr.org/papers/volume26/23-0671/23-0671.pdf
29. Improving performance of deep learning models with ... - NSF PAR, accessed March 17, 2026, https://par.nsf.gov/servlets/purl/10248707
30. Su-In Lee - Google Scholar, accessed March 17, 2026, https://scholar.google.com/citations?user=3ifikJ0AAAAJ&hl=en
31. Four Axiomatic Characterizations of the Integrated Gradients Attribution Method - Journal of Machine Learning Research, accessed March 17, 2026, https://www.jmlr.org/papers/volume26/23-0671/23-0671.pdf
32. Attribution Explanations for Deep Neural Networks: A Theoretical Perspective - arXiv, accessed March 17, 2026, https://arxiv.org/html/2508.07636v1
33. Incorporating Priors with Feature Attribution on Text Classification - ACL Anthology, accessed March 17, 2026, https://aclanthology.org/anthology-files/anthology-files/pdf/P/P19/P19-1631.pdf
34. Incorporating Priors with Feature Attribution on Text Classification - ACL Anthology, accessed March 17, 2026, https://aclanthology.org/P19-1631.pdf
35. arXiv:1906.08286v1 [cs.CL] 19 Jun 2019, accessed March 17, 2026, https://arxiv.org/pdf/1906.08286
36. Statsformer: Validated Ensemble Learning with LLM-Derived Semantic Priors - arXiv, accessed March 17, 2026, https://arxiv.org/html/2601.21410v1
37. Statsformer: Validated Ensemble Learning with LLM-Derived Semantic Priors - arXiv.org, accessed March 17, 2026, https://arxiv.org/abs/2601.21410
38. (PDF) Statsformer: Validated Ensemble Learning with LLM-Derived Semantic Priors, accessed March 17, 2026, https://www.researchgate.net/publication/400237885_Statsformer_Validated_Ensemble_Learning_with_LLM-Derived_Semantic_Priors
39. Erica Zhang - Google Scholar, accessed March 17, 2026, https://scholar.google.de/citations?user=GCxMlW0AAAAJ&hl=fr
40. Model Selection in Bayesian Neural Networks via Horseshoe Priors - Journal of Machine Learning Research, accessed March 17, 2026, https://jmlr.org/papers/volume20/19-236/19-236.pdf
41. Bayesian Neural Networks with Domain Knowledge Priors - OpenReview, accessed March 17, 2026, https://openreview.net/forum?id=Qa40qfZooj
42. Bayesian Neural Networks with Domain Knowledge Priors - arXiv.org, accessed March 17, 2026, https://arxiv.org/html/2402.13410v1
43. Bayesian neural networks for stock price forecasting before and during COVID-19 pandemic, accessed March 17, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC8248663/
44. Uncertainty-Aware Lookahead Factor Models for Quantitative Investing, accessed March 17, 2026, http://proceedings.mlr.press/v119/chauhan20a/chauhan20a.pdf
45. Uncertainty-Aware Lookahead Factor Models for Quantitative Investing, accessed March 17, 2026, https://ideas.repec.org/p/arx/papers/2007.04082.html
46. Trading with the Devil: Risk and Return in Foundation Model Strategies - arXiv,

accessed March 17, 2026, https://arxiv.org/html/2510.17165v1